01 — Einführung in Programmiersprachen

Was Computer verstehen – und wie wir mit ihnen reden

HTW Berlin – Wirtschaftsingenieurwesen
WiSe 2026/27

Agenda

1. Was ist eine Programmiersprache?
2. Sprachgenerationen: von Bitmustern zur Hochsprache
3. Python – unsere Wahl, und die Paradigmen-Landkarte
4. Interpreter vs. Compiler
5. Algorithmen – Schritt-für-Schritt-Denken
6. Bugs: Syntax-, Laufzeit-, Logikfehler
7. Syntax und Semantik
8. **Klassiker: Hello World** – live programmiert

Lernziel: Verstehen, *was* eine Programmiersprache ist und *was unter der Haube passiert*, wenn der Computer Python ausführt.

Heute: konzeptionelle Sitzung. Alle Demos zeige **ich** – *Python dient als Anschauung, die Sprache selbst lernen wir ab nächster Woche.* Ihre Vertiefung: Reflexionsfragen in Notebook 01.

Mensch und Maschine – zwei Welten

Aufgabe bei Veggie Soles: Tagesumsatz aus 1000 Bestellungen berechnen.

Ohne Programm

Von Hand: Stunden Arbeit, Flüchtigkeitsfehler, morgen wieder von vorn.

Mit Programm

Einmal die **Rechenvorschrift** aufschreiben – die Maschine erledigt sie in Millisekunden, heute, morgen, immer.

> Das Problem: die CPU (E4!) versteht nur **Bitmuster** – Menschen denken in Sätzen und Begriffen. Die Brücke dazwischen ist die **Programmiersprache**.

Was ist eine Programmiersprache?

Eine **formale Sprache**, mit der Menschen dem Computer Aufgaben beschreiben – aufgebaut wie jede Sprache:

In jeder Sprache gibt es…	…in Python heißt das
Vokabeln	Befehle wie `print`, `input`
Grammatik (**Syntax**)	wo Klammern und Anführungszeichen stehen
Bedeutung (**Semantik**)	was beim Ausführen tatsächlich passiert

Entscheidender Unterschied zu Deutsch: **formal** heißt, jede Regel ist exakt definiert – ein Tippfehler, und die Maschine versteht *gar nichts* mehr. Menschen raten mit, Computer nie.

Unter der Haube: Sprachgenerationen

Dieselbe Anweisung, drei Abstraktionsstufen:

Generation	so sieht sie aus	wer übersetzt sie?
1. Maschinencode	`10111000 00000001`	niemand – CPU führt direkt aus
2. Assembler	`MOV AX, 1`	Assembler (1 Befehl → 1 Bitmuster)
3. Hochsprache	`print("Hallo, Welt!")`	Compiler oder Interpreter

- Rückbezug E4: die CPU versteht **bis heute nur Bitmuster** – jede höhere Stufe ist Komfort für Menschen und muss **übersetzt** werden

Eine Zeile Hochsprache wird zu **vielen** Maschinenbefehlen – die Übersetzung leistet ein Programm, nicht Sie.

Warum Python?

- Anfang der 90er von Guido van Rossum entwickelt
- Name kommt von **Monty Python** – nicht von der Schlange
- Heute eine der **meistgenutzten** Sprachen weltweit (Google, NASA, Spotify, Forschung, KI)

Vier Eigenschaften, die Python anfängerfreundlich machen

- **Lesbar** – Code sieht oft fast wie Englisch aus
- **Wenig Boilerplate** – weniger merkwürdige Zeichen als z. B. Java oder C++
- **Vielseitig** – Web, Daten, KI, Automatisierung, Spiele
- **Riesige Community** – zu fast jeder Frage gibt es eine Antwort online

Die Paradigmen-Landkarte – wo liegt Python?

Ein **Paradigma** ist der Denkstil einer Sprache:

Paradigma	Leitfrage	Beispiele
imperativ	*Wie?* Schritt für Schritt	C, Pascal, **Python**
objektorientiert	Welche Objekte interagieren?	Java, C++, **Python**
funktional	Welche Funktionen bauen das Ergebnis?	Haskell, teils **Python**
deklarativ	*Was* soll herauskommen?	SQL, HTML

- Python ist **Multi-Paradigmen** – wir beginnen imperativ; die deklarative Welt (SQL) sehen Sie am Semesterende (NB 22). Heute nur Landkarte!

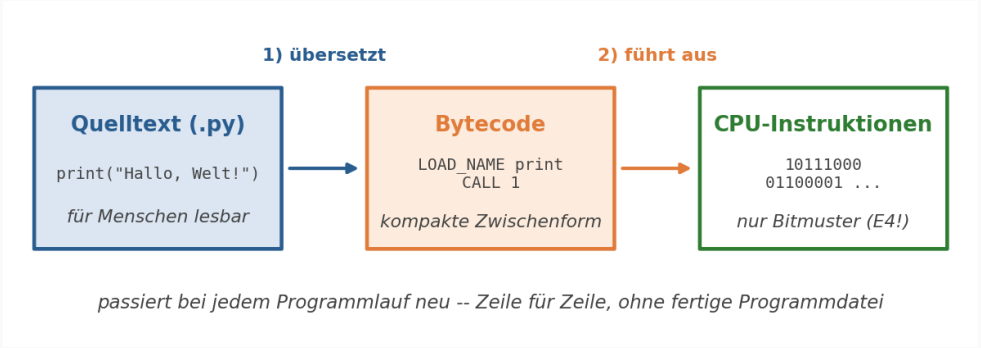
Wie führt der Computer Python aus?

```
print("Hallo, Welt!")
```

- Python ist eine **interpretierte Sprache**
- Der **Interpreter** übersetzt und führt Ihren Code **Zeile für Zeile** aus – kein Build-Schritt vorher
- Vorteil: sofort ausführbar, schnell ausprobieren, ideal zum Lernen

▶ **Live-Demo (nur ich)** – `01_hallo_welt.py` – *Python als Anschauung: die Sprache lernen wir ab nächster Woche.*

Unter der Haube: eine Zeile durch den Interpreter



Der Interpreter ist **selbst ein Programm**: er liest Ihre Zeile, übersetzt sie in kompakten **Bytecode** und lässt am Ende die CPU die zugehörigen **Bitmuster** ausführen (E4-Schichtenmodell). Es entsteht **keine** fertige Programmdatei – deshalb heißt es *zur Laufzeit*.

Interpreter vs. Compiler

	Interpretiert (Python, JS)	**Kompiliert** (C, C++, Rust)
Übersetzung	zur Laufzeit, Zeile für Zeile	einmal vorab, komplett
Ergebnis	keine Datei – direkt ausgeführt	ausführbare Datei (`.exe`)
Änderung testen	sofort	erst neu kompilieren
Geschwindigkeit	meist langsamer	meist schneller

▶ **Live-Demo (nur ich)** – `02_interpreter_zeile_fuer_zeile.py`: der Interpreter arbeitet stur von oben nach unten.

Unter der Haube: derselbe Tippfehler, zwei Welten

Zeile 2 enthält `Print` statt `print` – was passiert?

```
print("Willkommen bei Veggie Soles!")
Print("Heute im Angebot: Eco-Sneaker")
```

Schritt	Interpreter (Python)	Compiler (C)
vor dem Start	nichts zu tun	ganzes Programm übersetzen → **Fehler!**
Zeile 1	läuft, Ausgabe erscheint	wird nie ausgeführt
Zeile 2	Abbruch: `NameError`	– es entstand gar kein Programm

Konsequenz: interpretierte Programme **gründlich testen** – der Fehler in Zeile 200 fällt erst auf, wenn Zeile 200 wirklich läuft.

Wo schreibt man Python?

Werkzeug	Wann
Jupyter Notebook	interaktives Lernen, Daten, Mischung Text + Code
VSCode	richtige Skripte, Debugger, Terminal – *unser Klausur-Editor*
Texteditor (Notepad, …)	technisch möglich, aber ohne Hilfe
IDE (PyCharm, Spyder)	Profi-Setup, viele Funktionen

- In diesem Kurs: **Jupyter Notebooks** zum Lernen, **VSCode** zum Üben
- In der **Klausur** steht nur VSCode zur Verfügung

Was ist ein Algorithmus?

Eine **präzise Schritt-für-Schritt-Anleitung** zur Lösung eines Problems.

Algorithmus „Tee zubereiten”

1. Wasser in den Wasserkocher, einschalten
2. Teebeutel in die Tasse
3. Kochendes Wasser eingießen
4. 3–5 Minuten ziehen lassen
5. Beutel entfernen

Vom Problem zum Programm

Problem verstehen $\rightarrow$ Algorithmus entwerfen $\rightarrow$ Code schreiben $\rightarrow$ Testen $\rightarrow$ Optimieren

Unter der Haube: was macht eine Anleitung zum Algorithmus?

Nicht jedes Rezept genügt – die Informatik verlangt vier Eigenschaften:

- **Endlichkeit** – die Anleitung endet nach endlich vielen Schritten
- **Eindeutigkeit** – jeder Schritt ist präzise, kein Interpretationsspielraum („etwas Zucker" wäre keiner!)
- **Ausführbarkeit** – jeder Schritt ist tatsächlich durchführbar
- **Determinismus** – gleiche Eingabe $\rightarrow$ gleiches Ergebnis, bei jedem Lauf

Formal: ein Algorithmus berechnet eine Funktion $f : E \to A$ – er liefert für **jede** zulässige Eingabe $e \in E$ nach endlich vielen Schritten die Ausgabe $f(e)$.

Deshalb kann eine **Maschine** ihn ausführen: sie muss nie raten, nachfragen oder improvisieren.

Bugs – Fehler gehören dazu

- **Bug** = Fehler im Programm – benannt nach einer **echten Motte** in einem Computer der 1940er (Grace Hopper)
- **Auch Profis** machen ständig Fehler – der Unterschied: sie wissen, wie man sie systematisch findet

Art	Wann tritt er auf?	Schwierigkeitsgrad
Syntaxfehler	Code verletzt die Grammatik	leicht zu finden
Laufzeitfehler	während der Ausführung	mittel
Logikfehler	Code läuft, Ergebnis falsch	am schwersten

▶ **Live-Demo (nur ich)** – `03_syntax_fehler.py`: alle drei Fehlerklassen nacheinander provoziert.

Syntax: die Grammatik

Die **Syntax** legt fest, *wie* Code aussehen muss, damit Python ihn überhaupt versteht:

```
print("Hallo, Welt!")    # ok
Print("Hallo, Welt!")    # NameError: Print unbekannt
print "Hallo, Welt!"     # SyntaxError: Klammern fehlen
print("Hallo, Welt!      # SyntaxError: " nicht zu
```

- Welche Zeichen sind erlaubt? Wo stehen Klammern und Anführungszeichen?
- Python ist **case-sensitiv**: `print` $\neq$ `Print`

Bei Syntaxfehlern sagt Python meist **genau**, in welcher Zeile das Problem steckt – Fehlermeldungen lesen lohnt sich.

Semantik: die Bedeutung

Die **Semantik** beschreibt, *was* der Code tatsächlich tut, wenn er läuft.
Beide Sätze sind grammatikalisch korrekt – bedeuten aber Verschiedenes:

- „Der Hund beißt den Mann.“ / „Der Mann beißt den Hund.“

Beide Zeilen sind syntaktisch korrekt – aber nur eine ist *richtig*:

```
print("Eco-Sneaker kostet 89.95 EUR")
print("Eco-Sneaker kostet 98.95 EUR")   # Tippdreher!
```

Syntaktisch korrekt $\neq$ semantisch korrekt. Code, der läuft, ist nicht automatisch *richtig*.

Unter der Haube: was die Maschine erkennt – und was nicht

Vor dem Ausführen prüft der **Parser** jede Zeile gegen die Grammatikregeln – Ihre **Absicht** dagegen steht nirgends im Code:

Fehler	Beispiel	Wer findet ihn?
Syntax	`print("Hallo`	Parser, **vor** dem Lauf, mit Zeilennummer
Laufzeit	`Print(...)`	Interpreter, **beim** Ausführen
Logik	`98.95` statt `89.95`	**nur Sie** – die Maschine merkt nichts

Merksatz: die Maschine prüft die **Form**, nie den **Sinn**. Je später ein Fehler auffällt, desto teurer ist er – Logikfehler findet kein Computer für Sie.

Klassiker: Hello World – die Aufgabe

Das erste Programm der Informatikgeschichte(n): seit Kernighan & Ritchie (1978) beginnt jede Programmierkarriere mit diesem Gruß.

- Eine Datei `hallo.py` mit **einer einzigen Zeile** Code
- Beim Ausführen erscheint exakt der Gruß unten
- Danach: Gruß **personalisieren** – Text ändern, erneut ausführen

So soll es laufen

```
Hallo, Welt!
```

> Klingt winzig – ist aber der Moment, in dem Ihr Rechner zum ersten Mal *Ihr* Programm ausführt: Quelltext → Interpreter → Bitmuster → Ausgabe.

Klassiker: Hello World – Live-Coding

▶ **Live-Coding ab leerem Editor** – Referenz: `90_klassiker_hello_world.py`
Reine Dozenten-Demo: Sie schauen zu – `print()` lernen wir formal ab nächster Woche (Notebook 02).
Teilziele:

1. VSCode öffnen, neue Datei `hallo.py` anlegen
2. Die eine Zeile schreiben: `print` + Klammern + Anführungszeichen
3. Ausführen – der Interpreter antwortet mit dem Gruß
4. Gruß personalisieren und erneut ausführen

✎ **Zum Nachmachen** – Handout *“Klassiker 01: Hello World”* (`Klassiker/01_Klassiker_Hello_World.pdf`): Schritt-für-Schritt-Anleitung zum Abtippen, kein Vorwissen nötig.

Cheat Card

Begriff	Worum geht’s?
Programmiersprache	formale Brücke zwischen Mensch und Maschine
Sprachgenerationen	Maschinencode → Assembler → Hochsprache
Interpreter	übersetzt zur Laufzeit, Zeile für Zeile (Python)
Compiler	übersetzt einmal vorab in eine ausführbare Datei
Paradigma	Denkstil: imperativ, OO, funktional, deklarativ
Algorithmus	endliche, eindeutige Schritt-für-Schritt-Anleitung
Syntax	Form – prüft die Maschine (Parser)
Semantik	Bedeutung – prüfen nur Sie
Klassiker heute	Hello World: eine Zeile `print("Hallo, Welt!")`

Faustregel: „läuft” heißt nur *Syntax ok* – „richtig” heißt *Semantik ok*. Den Unterschied prüft keine Maschine für Sie.

Ausblick: Notebook 02 – Konsole

Unser Hello World hat eine Schwäche: es grüßt **immer gleich** – das Programm kann nicht zuhören.

```
print("Willkommen bei Veggie Soles!")
input("Wie heissen Sie? ")
```

- `print(...)` – der **Mund** Ihres Programms (ab nächster Woche Ihr Werkzeug)
- `input(...)` – das **Ohr**: ab jetzt reagieren Programme auf Eingaben

Nächste Sitzung schreiben **Sie selbst** den ersten Dialog mit dem Computer.

Heute geübt

- ✓ Programmiersprache: formale Brücke Mensch – Maschine
- ✓ Sprachgenerationen: die CPU versteht nur Bitmuster
- ✓ Paradigmen-Landkarte: Python ist Multi-Paradigmen
- ✓ Interpreter vs. Compiler; Quelltext $\rightarrow$ Bytecode $\rightarrow$ CPU
- ✓ Algorithmus: endlich, eindeutig, ausführbar, deterministisch
- ✓ Fehlerklassen: Syntax prüft die Maschine, Semantik nur Sie
- ✓ **Klassiker Hello World** live gesehen

✎ **Ihre Aufgaben bis zur nächsten Woche:**

- Klassiker **Hello World** nachmachen (Handout, reines Abtippen)
- Reflexionsfragen Notebook 01 (Syntax vs. Semantik in eigenen Worten)
- Jupyter + VSCode lauffähig machen – ab NB 02 schreiben Sie Code